

NeuroSky MindWave Windows SDK

Developer Guide · v2.0.0

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Installation](#)
 4. [Quick Start](#)
 5. [Connection Modes](#)
 6. [EEG Data Model](#)
 7. [EEG Frequency Bands](#)
 8. [Signal Quality](#)
 9. [Commands](#)
 10. [Simulator](#)
 11. [Error Handling](#)
 12. [Advanced Patterns](#)
 13. [Finding Your Device Address](#)
 14. [Troubleshooting](#)
 15. [API Reference](#)
-

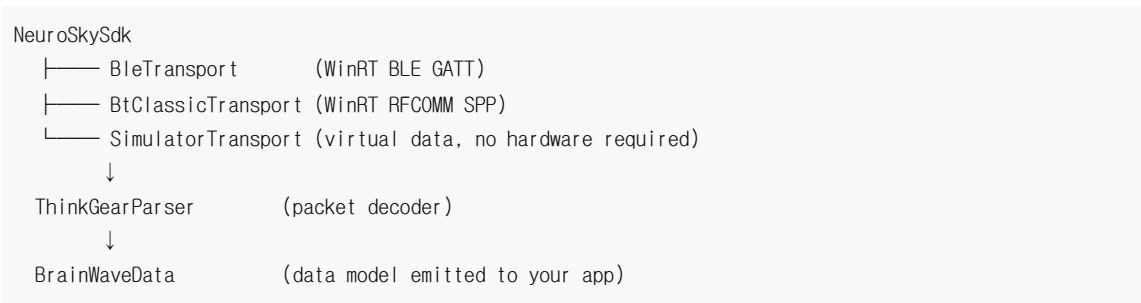
1. Overview

The NeuroSky MindWave Windows SDK is a modern C# library for reading EEG data from NeuroSky MindWave Mobile headsets on Windows 10+.

Key features:

- No TGC (ThinkGear Connector) dependency — pure WinRT Bluetooth
- BLE GATT + BT Classic RFCOMM support
- Developer-selectable transport: Auto / BLE only / BT Classic only
- `IAsyncEnumerable<BrainWaveData>` stream API — works natively with `await foreach`
- Built-in Simulator for development without a real device
- .NET 8, C# 12, fully nullable-annotated

Architecture overview:



2. Requirements

Requirement	Minimum
OS	Windows 10 version 1903 (build 18362)
.NET	.NET 8.0
Bluetooth	BLE adapter (for BLE mode) or Classic BT adapter (for BT Classic mode)
Pairing	Not required for BLE; required for BT Classic

Note: The SDK targets `net8.0-windows10.0.19041.0`. This enables WinRT APIs (`Windows.Devices.Bluetooth`) directly from .NET without a UWP container.

3. Installation

NuGet Package Manager (Visual Studio)

```
Tools → NuGet Package Manager → Manage NuGet Packages
Search: NeuroSky.MindWave.Sdk
Install
```

.csproj (recommended)

```
<PackageReference Include="NeuroSky.MindWave.Sdk" Version="2.0.0" />
```

.NET CLI

```
dotnet add package NeuroSky.MindWave.Sdk
```

Your project must also target `net8.0-windows10.0.19041.0` or later:

```
<TargetFramework>net8.0-windows10.0.19041.0</TargetFramework>
```

4. Quick Start

```
using NeuroSky.Sdk;
using NeuroSky.Sdk.Transport;

// 1. Create SDK instance
await using var sdk = new NeuroSkySdk();

// 2. Subscribe to connection state changes (optional)
sdk.StateChanged += (_, state) =>
    Console.WriteLine($"[State] {state}");

// 3. Set up graceful cancellation
using var cts = new CancellationTokenSource();
```

```

Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };

// 4. Connect (Auto mode: BLE first, BT Classic fallback)
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");

// 5. Stream EEG data
await foreach (var data in sdk.DataStream(cts.Token))
{
    Console.WriteLine($"Attention : {data.Attention}");
    Console.WriteLine($"Meditation : {data.Meditation}");
    Console.WriteLine($"Signal : {data.SignalQuality}");
    Console.WriteLine($"Delta : {data.Delta}");
}

```

Replace "AA:BB:CC:DD:EE:FF" with your headset's Bluetooth MAC address. See [Section 13](#) for how to find it.

5. Connection Modes

The `TransportMode` enum gives you explicit control over which Bluetooth transport to use.

```

public enum TransportMode
{
    Auto,          // BLE first → BT Classic fallback (default)
    Ble,           // BLE only
    BtClassic      // BT Classic only
}

```

TransportMode.Auto (default)

```

await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");
// same as:
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.Auto);

```

Behavior:

1. Attempts BLE connection
2. If not connected within **5 seconds**, automatically switches to BT Classic
3. The `DataStream` API works identically regardless of which transport wins

Use this mode when you want maximum compatibility without caring about the underlying transport.

TransportMode.Ble

```

await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.Ble);

```

- Uses WinRT BLE GATT (`Windows.Devices.Bluetooth.GenericAttributeProfile`)
- **No Windows pairing required** — the SDK connects directly
- Lower power consumption than BT Classic

- Requires a BLE-capable Bluetooth adapter

Use this mode when you know the device supports BLE and want to avoid the pairing requirement.

TransportMode.BtClassic

```
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.BtClassic);
```

- Uses WinRT RFCOMM SPP (Windows.Devices.Bluetooth.Rfcomm)
- **Requires the device to be paired first** in Windows Bluetooth settings
- More stable in environments with BLE interference
- SPP UUID: 00001101-0000-1000-8000-00805f9b34fb

Pairing steps:

1. Settings → Bluetooth & other devices
2. Add device → Bluetooth
3. Select MindWave Mobile and complete pairing

Connection state flow

```
Disconnected → Scanning → Connecting → Connected
                                   ↘ Error
```

Subscribe to StateChanged to track transitions:

```
sdk.StateChanged += (_, state) =>
{
    switch (state)
    {
        case ConnectionState.Scanning: Console.WriteLine("Scanning..."); break;
        case ConnectionState.Connecting: Console.WriteLine("Connecting..."); break;
        case ConnectionState.Connected: Console.WriteLine("Connected!"); break;
        case ConnectionState.Error: Console.WriteLine("Connection error"); break;
    }
};
```

6. EEG Data Model

Every item yielded by DataStream() is a BrainWaveData record:

```
public record BrainWaveData
{
    public long Timestamp { get; init; } // Unix ms (UTC)
    public int PoorSignal { get; init; } // 0 = perfect, 200 = no signal
    public int Attention { get; init; } // 0~100
    public int Meditation { get; init; } // 0~100
    public int Delta { get; init; } // 0.5~2.75 Hz
    public int Theta { get; init; } // 3.5~6.75 Hz
}
```

```

public int LowAlpha { get; init; } // 7.5~9.25 Hz
public int HighAlpha { get; init; } // 10~11.75 Hz
public int LowBeta { get; init; } // 13~16.75 Hz
public int HighBeta { get; init; } // 18~29.75 Hz
public int LowGamma { get; init; } // 31~39.75 Hz
public int MidGamma { get; init; } // 41~49.75 Hz
public IReadOnlyList<int> RawEeg { get; init; } // 512Hz, 10 samples/packet
public int EyeBlink { get; init; } // 0~255, blink intensity
public SignalQuality SignalQuality { get; } // derived enum
}

```

Data rates

Data type	Rate	Notes
Attention / Meditation	~1 Hz	One value per second
EEG frequency bands	~1 Hz	Delta through MidGamma
Raw EEG	512 Hz	10 samples per packet (~51 packets/sec)
Eye blink	Event-driven	Only present when blink detected

7. EEG Frequency Bands

The ThinkGear chip outputs 8 frequency band power values. Units are arbitrary (relative power).

Property	Band	Frequency	Associated mental states
Delta	δ Delta	0.5~2.75 Hz	Deep sleep, healing
Theta	θ Theta	3.5~6.75 Hz	Drowsiness, creativity, REM
LowAlpha	α Low Alpha	7.5~9.25 Hz	Relaxed, calm
HighAlpha	α High Alpha	10~11.75 Hz	Relaxed, eyes-closed rest
LowBeta	β Low Beta	13~16.75 Hz	Focused, alert
HighBeta	β High Beta	18~29.75 Hz	Anxiety, excitement
LowGamma	γ Low Gamma	31~39.75 Hz	Higher cognition, perception
MidGamma	γ Mid Gamma	41~49.75 Hz	Intense focus, binding

Attention and Meditation (eSense™)

NeuroSky's proprietary eSense™ algorithm combines multiple frequency bands to produce single values:

- **Attention (0~100):** Higher values indicate stronger active mental focus. Values below 40 are considered low; 60+ is moderate; 80+ is high.
- **Meditation (0~100):** Higher values indicate calmer, more relaxed mental states. Correlated with Alpha activity.

These are proprietary processed values, not raw frequency powers.

Raw EEG

```
foreach (var sample in data.RawEeg) // 10 samples per packet
{
    // Each sample is a signed 16-bit integer: -32768 to +32767
    // Sample rate: 512 Hz
    Console.WriteLine(sample);
}
```

8. Signal Quality

PoorSignal (0–200) indicates electrode contact quality:

Value	SignalQuality	Meaning
0	Good	Perfect contact
1~50	Fair	Acceptable signal
51~199	Poor	Poor contact — reposition headset
200	NoSignal	No contact / not wearing

```
await foreach (var data in sdk.DataStream(cts.Token))
{
    if (data.SignalQuality == SignalQuality.NoSignal)
    {
        Console.WriteLine("Please put on the headset.");
        continue;
    }
    if (data.SignalQuality == SignalQuality.Poor)
    {
        Console.WriteLine("Weak signal - adjust the sensor.");
    }

    // Use data only when signal is Fair or Good
    ProcessData(data);
}
```

9. Commands

Send control commands to the headset after connecting:

```
// Notch filter - removes power-line noise
// Use 60Hz for Korea, USA, Canada, Mexico, Japan
await sdk.SendCommandAsync(NeuroSkyCommand.Notch60Hz);
```

```
// Use 50Hz for China, Europe, Australia, UK
await sdk.SendCommandAsync(NeuroSkyCommand.Notch50Hz);

// Raw EEG streaming
await sdk.SendCommandAsync(NeuroSkyCommand.StartRawEeg);
await sdk.SendCommandAsync(NeuroSkyCommand.StopRawEeg);
```

All commands

Constant	Byte	Description
Notch60Hz	0x1C	Notch filter at 60 Hz (Korea/USA)
Notch50Hz	0x1B	Notch filter at 50 Hz (China/Europe)
StartRawEeg	0x15	Enable raw EEG stream
StopRawEeg	0x16	Disable raw EEG stream
StartEsense	0x17	Enable eSense (Attention/Meditation)
StopEsense	0x18	Disable eSense

Tip: Raw EEG is disabled by default. Call `StartRawEeg` after connecting if you need it.

10. Simulator

Use `SimulatorTransport` to develop and test your application without a real MindWave headset.

```
using NeuroSky.Sdk.Simulator;

var simulator = new SimulatorTransport();
simulator.SetMode(SimulatorTransport.Mode.Focused);

await simulator.ConnectAsync("simulator");

await foreach (var data in simulator.DataStream(cts.Token))
{
    Console.WriteLine($"Attention: {data.Attention}");
}
```

Simulator modes

Mode	Attention	Meditation	Use case
Random	0~100 (random)	0~100 (random)	General integration testing
Focused	70~100	40~60	Focused-state UI testing
Relaxed	20~50	70~100	Relaxed-state UI testing
PoorSignal	0	0	Signal loss / error handling testing

Using Simulator with dependency injection

```
// In tests or development builds, inject SimulatorTransport
// instead of calling sdk.ConnectAsync with a real device

ITransport transport = isDevelopment
    ? new SimulatorTransport()
    : sdk; // NeuroSkySdk also implements ITransport
```

11. Error Handling

Connection errors

```
try
{
    await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.Ble);
}
catch (OperationCanceledException)
{
    Console.WriteLine("Connection timed out or was cancelled.");
}
catch (Exception ex)
{
    Console.WriteLine($"Connection failed: {ex.Message}");
}
```

Stream errors

The `DataStream` will stop if the connection drops. Handle this by wrapping in a retry loop:

```
while (!cts.Token.IsCancellationRequested)
{
    try
    {
        await sdk.ConnectAsync(address);

        await foreach (var data in sdk.DataStream(cts.Token))
        {
            ProcessData(data);
        }
    }
    catch (OperationCanceledException) when (cts.Token.IsCancellationRequested)
    {
        break; // User cancelled - clean exit
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Disconnected: {ex.Message}. Reconnecting in 3s...");
    }
}
```



```
        await Task.Delay(3000, cts.Token);
    }
}
```

12. Advanced Patterns

WPF / MAUI integration

```
// ViewModel
public class MainViewModel : IAsyncDisposable
{
    private readonly NeuroSkySdk _sdk = new();
    private CancellationTokenSource _cts = new();

    public int Attention { get; private set; }
    public int Meditation { get; private set; }
    public string SignalStatus { get; private set; } = "Disconnected";

    public async Task StartAsync(string address)
    {
        _sdk.StateChanged += (_, state) =>
        {
            SignalStatus = state.ToString();
            OnPropertyChanged(nameof(SignalStatus));
        };

        await _sdk.ConnectAsync(address);

        _ = Task.Run(async () =>
        {
            await foreach (var data in _sdk.DataStream(_cts.Token))
            {
                Attention = data.Attention;
                Meditation = data.Meditation;
                OnPropertyChanged(nameof(Attention));
                OnPropertyChanged(nameof(Meditation));
            }
        });
    }

    public async ValueTask DisposeAsync()
    {
        _cts.Cancel();
        await _sdk.DisposeAsync();
    }
}
```

Buffering raw EEG samples

```

var buffer = new List<int>();

await foreach (var data in sdk.DataStream(cts.Token))
{
    buffer.AddRange(data.RawEeg); // 10 samples per packet

    if (buffer.Count >= 512) // 1 second of data at 512 Hz
    {
        AnalyzeOneSecond(buffer.Take(512).ToArray());
        buffer.RemoveRange(0, 512);
    }
}

```

Recording to CSV

```

await using var writer = new StreamWriter("eeg_recording.csv");
await writer.WriteLineAsync(
    "timestamp,attention,meditation,poorSignal,delta,theta," +
    "lowAlpha,highAlpha,lowBeta,highBeta,lowGamma,midGamma");

await foreach (var d in sdk.DataStream(cts.Token))
{
    await writer.WriteLineAsync(
        $"{d.Timestamp},{d.Attention},{d.Meditation},{d.PoorSignal}," +
        $"{d.Delta},{d.Theta},{d.LowAlpha},{d.HighAlpha}," +
        $"{d.LowBeta},{d.HighBeta},{d.LowGamma},{d.MidGamma}");
}

```

13. Finding Your Device Address

Windows Settings

Settings → Bluetooth & other devices
 → Click "MindWave Mobile"
 → "More info" or "Properties"
 → Copy the MAC address shown

PowerShell

```

# Find all Bluetooth devices with "MindWave" in the name
Get-PnpDevice -Class Bluetooth |
    Where-Object { $_.FriendlyName -like "*MindWave*" } |
    Select-Object FriendlyName, DeviceID

```

From the DeviceID output

DeviceID looks like:

```
BTHENUMW{00001101-0000-1000-8000-00805F9B34FB}_LOCALMFG&0002W7&...&AA_BB_CC_DD_EE_FF&...
```

The last part `AA_BB_CC_DD_EE_FF` is the MAC address — replace `_` with `:`.

14. Troubleshooting

Symptom	Cause	Fix
<code>ConnectionState.Error</code> immediately	BLE adapter not found	Check Device Manager for Bluetooth adapter
Timeout in Auto mode, falls back to BT	Normal fallback behavior	Either pair the device, or use <code>TransportMode.Ble</code>
<code>BtClassic</code> fails with access denied	Device not paired	Pair in Settings → Bluetooth & other devices
<code>SignalQuality.NoSignal</code> always	Sensor not touching skin	Wet the sensor contact point, adjust headset
Attention and Meditation always 0	eSense not started	Ensure signal quality is Fair or Good first
Raw EEG empty (<code>RawEeg.Count == 0</code>)	Raw EEG not enabled	Call <code>SendCommandAsync(NeuroSkyCommand.StartRawEeg)</code> after connecting
High noise in raw EEG	Power-line interference	Set the correct notch filter (50Hz or 60Hz)
App crashes with <code>PlatformNotSupportedException</code>	Running on non-Windows target	Ensure <code>TargetFramework</code> includes <code>– windows10.0.19041.0</code>

15. API Reference

NeuroSkySdk

```
public sealed class NeuroSkySdk : IDisposable
```

Member	Description
<code>ConnectionState State</code>	Current connection state
<code>event EventHandler<ConnectionState> StateChanged</code>	Fired on state transitions
<code>Task ConnectAsync(string address, TransportMode mode, CancellationToken ct)</code>	Connect to a headset
<code>Task DisconnectAsync()</code>	Disconnect the active transport

IAsyncEnumerable<BrainWaveData> DataStream(CancellationToken ct)	Real-time EEG data stream
Task SendCommandAsync(byte cmd)	Send a control command
ValueTask DisposeAsync()	Dispose and disconnect

BrainWaveData

Property	Type	Range	Description
Timestamp	long	Unix ms	UTC time of packet reception
PoorSignal	int	0~200	Electrode contact quality
Attention	int	0~100	eSense attention level
Meditation	int	0~100	eSense meditation level
Delta	int	0~∞	Delta band power (0.5~2.75 Hz)
Theta	int	0~∞	Theta band power (3.5~6.75 Hz)
LowAlpha	int	0~∞	Low alpha power (7.5~9.25 Hz)
HighAlpha	int	0~∞	High alpha power (10~11.75 Hz)
LowBeta	int	0~∞	Low beta power (13~16.75 Hz)
HighBeta	int	0~∞	High beta power (18~29.75 Hz)
LowGamma	int	0~∞	Low gamma power (31~39.75 Hz)
MidGamma	int	0~∞	Mid gamma power (41~49.75 Hz)
RawEeg	IReadOnlyList<int>	-32768~32767	512Hz raw ADC values
EyeBlink	int	0~255	Eye blink intensity
SignalQuality	SignalQuality	enum	Derived from PoorSignal

TransportMode

Value	Behavior
Auto	BLE first; BT Classic fallback after 5 seconds
Ble	BLE only (WinRT BLE GATT)
BtClassic	BT Classic only (WinRT RFCOMM SPP)

ConnectionState

Value	Meaning
Disconnected	Not connected

Scanning	Searching for device
Connecting	Establishing connection
Connected	Ready to stream data
Error	Connection failed

SignalQuality

Value	PoorSignal range	Meaning
Good	0	Perfect contact
Fair	1~50	Usable signal
Poor	51~199	Weak contact
NoSignal	200	No contact

NeuroSkyCommand

Constant	Byte	Description
Notch60Hz	0x1C	Notch filter 60 Hz
Notch50Hz	0x1B	Notch filter 50 Hz
StartRawEeg	0x15	Enable raw EEG
StopRawEeg	0x16	Disable raw EEG
StartEsense	0x17	Enable eSense
StopEsense	0x18	Disable eSense